StandardComposer 분석

1. StandardComposer의 구조와 역할

StandardComposer는 PLONK 회로를 구성하기 위한 핵심 클래스입니다. 이 클래스는 회로의 제약 조건을 정의하고, 변수를 관리하며, 다양한 게이트를 추가하는 기능을 제공합니다.
StandardComposer는 일종의 회로 빌더(Circuit Builder)로, 사용자가 회로를 구성하고 이를 증명 시스템에서 사용할 수 있는 형태로 변환하는 역할을 합니다.

1.0 핵심 역할

StandardComposer의 핵심 역할은 다음과 같습니다:

1. **회로 구성**: 다양한 게이트와 제약 조건을 추가하여 회로를 구성합니다.
2. **변수 관리**: 회로 내의 변수들을 생성하고 관리합니다.
3. **선택자 관리**: 각 게이트 타입에 대한 선택자 벡터를 관리합니다.
4. **순열 관리**: 변수 간의 등가성을 나타내는 순열 인수를 관리합니다.
5. **공개 입력 관리**: 회로의 공개 입력을 관리합니다.
6. **룩업 테이블 관리**: 효율적인 연산을 위한 룩업 테이블을 관리합니다.

1.1 주요 필드

StandardComposer의 주요 필드는 다음과 같습니다:

```
pub struct StandardComposer<F, P>
where
F: PrimeField,
```

```
    P: ModelParameters<BaseField = F>,

    {

    // 회로의 게이트 수

    pub(crate) n: usize,



    // 선택자 벡터

    pub(crate) q_m: Vec<F>, // 곱셈 선택자

    pub(crate) q_l: Vec<F>, // 왼쪽 와이어 선택자

    pub(crate) q_r: Vec<F>, // 오른쪽 와이어 선택자

    pub(crate) q_o: Vec<F>, // 출력 와이어 선택자

    pub(crate) q_4: Vec<F>, // 네 번째 와이어 선택자

    pub(crate) q_c: Vec<F>, // 상수 선택자

    // 고차 선택자 (Poseidon 해시 등에 사용)

    pub(crate) q_hl: Vec<F>, // w_l^5 선택자

    pub(crate) q_hr: Vec<F>, // w_r^5 선택자

    pub(crate) q_h4: Vec<F>, // w_4^5 선택자

    // 게이트 타입 선택자

    pub(crate) q_arith: Vec<F>, // 산술 게이트 선택자

    pub(crate) q_range: Vec<F>, // 범위 게이트 선택자

    pub(crate) q_logic: Vec<F>, // 논리 게이트 선택자

    pub(crate) q_fixed_group_add: Vec<F>, // 고정 기반 그룹 덧셈 선택자

    pub(crate) q_variable_group_add: Vec<F>, // 가변 기반 그룹 덧셈 선택자

    pub(crate) q_lookup: Vec<F>, // 룩업 게이트 선택자
```

```
    // 공개 입력
    pub(crate) public_inputs: PublicInputs<F>,
    pub(crate) intended_pi_pos: Vec<usize>,

    // 와이어 벡터
    pub(crate) w_l: Vec<Variable>, // 왼쪽 와이어
    pub(crate) w_r: Vec<Variable>, // 오른쪽 와이어
    pub(crate) w_o: Vec<Variable>, // 출력 와이어
    pub(crate) w_4: Vec<Variable>, // 네 번째 와이어

    // 룩업 테이블
    pub(crate) lookup_table: LookupTable<F>,

    // 0 값을 가진 변수
    pub(crate) zero_var: Variable,

    // 변수 값 맵
    pub(crate) variables: HashMap<Variable, F>,

    // 순열 인수
    pub(crate) perm: Permutation,
```

```
// 타입 파라미터 마커

__: PhantomData<P>,

}
```

1.2 주요 메서드

StandardComposer의 주요 메서드는 다음과 같습니다:

1.2.1 생성자 및 초기화 메서드

- `new()` : 빈 StandardComposer를 생성합니다. 내부적으로 `with_expected_size(0)` 을 호출합니다.
- `with_expected_size(expected_size: usize)` : 예상 회로 크기로 StandardComposer를 생성합니다. 이는 메모리 재할당을 줄이기 위해 벡터들을 미리 할당합니다.
- `add_blinding_factors<R>(&mut self, rng: &mut R)` : 블라인딩 팩터를 추가하여 영지식 속성을 보장합니다.

1.2.2 변수 관리 메서드

- `add_input(&mut self, s: F) -> Variable` : 새 변수를 추가하고 값을 연결합니다. 순열 시스템에서 새 변수 ID를 가져와 `variables` 맵에 저장합니다.
- `add_witness_to_circuit_description(&mut self, value: F) -> Variable` : 회로 설명에 고정된 변수를 추가합니다. 이 변수는 상수로 제약됩니다.
- `zero_var(&self) -> Variable` : 0 값을 가진 특수 변수를 반환합니다. 이는 4번째 와이어가 사용되지 않을 때 기본값으로 사용됩니다.

1.2.3 게이트 추가 메서드

- `poly_gate(...)` : 가장 일반적인 게이트로, 모든 선택자 값을 직접 지정할 수 있습니다.
- `arithmetic_gate<Fn>(&mut self, func: Fn) -> Variable` : 산술 게이트를 추가합니다. 클로저를 통해 게이트 속성을 설정합니다.

- `range_gate(&mut self, a: Variable, num_bits: usize) -> Vec<Variable>` : 변수가 지정된 비트 수 범위 내에 있는지 확인하는 게이트를 추가합니다.

- `boolean_gate(&mut self, a: Variable) -> Variable` : 변수가 0 또는 1인지 확인하는 게이트를 추가합니다.

- `logic_gate(&mut self, a: Variable, b: Variable, op: LogicOp) -> Variable` : 논리 연산(AND, XOR 등)을 수행하는 게이트를 추가합니다.

1.2.4 제약 조건 추가 메서드

- `constrain_to_constant(&mut self, a: Variable, constant: F, pi: Option<F>)` : 변수를 특정 상수 값으로 제약합니다.

- `assert_equal(&mut self, a: Variable, b: Variable)` : 두 변수가 같은 값을 가지도록 제약합니다.

1.2.5 조건부 선택 메서드

- `conditional_select(&mut self, bit: Variable, choice_a: Variable, choice_b: Variable) -> Variable` : 비트 값에 따라 두 변수 중 하나를 선택합니다.

- `conditional_select_zero(&mut self, bit: Variable, value: Variable) -> Variable` : 비트가 1이면 value, 0이면 0을 반환합니다.

- `conditional_select_one(&mut self, bit: Variable, value: Variable) -> Variable` : 비트가 1이면 value, 0이면 1을 반환합니다.

- `is_zero_with_output(&mut self, a: Variable) -> Variable` : 변수가 0인지 확인하고 불리언 결과를 반환합니다.

- `is_eq_with_output(&mut self, a: Variable, b: Variable) -> Variable` : 두 변수가 같은지 확인하고 불리언 결과를 반환합니다.

1.2.6 유틸리티 메서드

- `total_size(&self) -> usize` : 회로의 총 크기를 반환합니다. 룩업 테이블 크기와 게이트 수 중 큰 값을 반환합니다.

- `circuit_bound(&self) -> usize` : 회로의 경계(다음 2의 거듭제곱)를 반환합니다.

- `get_pi(&self) -> &PublicInputs<F>` : 공개 입력을 반환합니다.
- `add_pi<T>(&mut self, pos: usize, item: &T) -> Result<(), Error>` : 특정 위치에 공개 입력을 추가합니다.
- `check_circuit_satisfied(&mut self)` : 회로가 만족되는지 확인합니다 (디버그 기능).

2. 회로 구성 방법

StandardComposer를 사용한 회로 구성 방법은 다음과 같습니다:

2.1 회로 초기화

회로를 초기화하는 방법은 다음과 같습니다:

```
// 빈 회로 생성
let mut composer = StandardComposer::<BlsScalar, JubJubParameters>::new();

// 예상 크기로 회로 생성
let mut composer = StandardComposer::<BlsScalar, JubJubParameters>::with_expec
```

2.2 변수 추가

변수를 추가하는 방법은 다음과 같습니다:

```
// 변수 추가
let a = composer.add_input(BlsScalar::from(20u64));
let b = composer.add_input(BlsScalar::from(5u64));
```

```
// 0 변수 가져오기

let  zero = composer.zero_var();
```

2.3 게이트 추가

다양한 게이트를 추가하는 방법은 다음과 같습니다:

2.3.1 산술 게이트

산술 게이트는 다음과 같은 형태의 제약 조건을 추가합니다:

```
(a * b) * q_m + a * q_l + b * q_r + q_c + PI + q_o * c + q_4 * d = 0
```

```
// 산술 게이트 추가 (a + b = c)

let  c = composer.arithmetic_gate(|gate| {

gate.witness(a, b, None)

.add(F::one(), F::one())

});
```

```
// 산술 게이트 추가 (a * b = c)

let  c = composer.arithmetic_gate(|gate| {

gate.witness(a, b, None)

.mul(F::one())

});
```

```
// 산술 게이트 추가 (a + b = c, 공개 입력 포함)
```

```
let  c = composer.arithmetic_gate(|gate| {

gate.witness(a, b, None)

.add(F::one(), F::one())

.pi(-expected_sum)

});
```

2.3.2 다항식 게이트

다항식 게이트는 산술 게이트의 일반화된 형태로, 사용자가 모든 선택자를 직접 지정할 수 있습니다:

```
// 다항식 게이트 추가

let (a, b, c) = composer.poly_gate(

a, b, c,

q_m, q_l, q_r, q_o, q_c,

pi,

);
```

2.3.3 제약 조건 추가

변수에 제약 조건을 추가하는 방법은 다음과 같습니다:

```
// 변수를 상수로 제약

composer.constrain_to_constant(a, F::from(20u64), None);



// 두 변수가 같음을 제약
```

```
composer.assert_equal(a, b);
```

2.3.4 조건부 선택

조건부 선택을 수행하는 방법은 다음과 같습니다:

```
// 비트 값에 따라 변수 선택
let result = composer.conditional_select(bit, choice_a, choice_b);

// 비트 값이 1이면 value, 0이면 0 반환
let result = composer.conditional_select_zero(bit, value);

// 비트 값이 1이면 value, 0이면 1 반환
let result = composer.conditional_select_one(bit, value);
```

3. 변수 관리 방식

StandardComposer는 변수를 다음과 같은 방식으로 관리합니다:

3.1 변수 표현

변수는 `Variable` 구조체로 표현됩니다:

```
#[derive(Clone, Copy, Debug, Eq, Hash, PartialEq)]
pub struct Variable(pub(crate) usize);
```

이 구조체는 단순한 정수 식별자를 래핑하고 있으며, 이 식별자는 변수의 고유 ID로 사용됩니다. 변수는 복사 가능하고 해시 가능하며, 등가성 비교가 가능합니다.

3.2 변수 생성 과정

변수는 `add_input` 메서드를 통해 생성됩니다:

```
pub fn add_input(&mut self, s: F) -> Variable {
// 순열에서 새 변수 가져오기
let var = self.perm.new_variable();
// 변수를 값에 연결
self.variables.insert(var, s);


var
}
```

이 메서드는 다음과 같은 작업을 수행합니다:

1. `Permutation::new_variable()` 을 호출하여 새 변수 ID를 생성합니다.
2. 변수 ID와 실제 값을 `variables` HashMap에 저장합니다.
3. 순열 시스템에서 변수를 위한 공간을 할당합니다.
4. 생성된 변수 ID를 반환합니다.

3.3 순열 시스템과의 연동

`Permutation` 구조체는 변수 관리의 핵심 역할을 합니다:

```
impl Permutation {
```

```
pub fn new_variable(&mut self) -> Variable {

// 변수 ID 생성 (현재 변수 맵 크기 기반)

let var = Variable(self.variable_map.keys().len());

// 변수를 위한 와이어 데이터 벡터 할당

self.variable_map.insert(var, Vec::with_capacity(16));

var

}

}
```

각 변수는 `variable_map` 에서 `Vec<WireData>` 와 연결되어, 해당 변수가 사용되는 모든 와이어 위치를 추적합니다.

3.4 변수 값 접근

변수의 값은 `variables` HashMap을 통해 접근할 수 있습니다:

```
let value = self.variables.get(&var).unwrap();

// 또는 안전한 접근

if let Some(value) = self.variables.get(&var) {

// 값 사용

}
```

3.5 특수 변수

StandardComposer는 다음과 같은 특수 변수를 관리합니다:

3.5.1 Zero Variable

- `zero_var` : 0 값을 가진 변수로, 회로 초기화 시 생성됩니다.
- 4번째 와이어가 사용되지 않는 게이트에서 기본값으로 사용됩니다.

```
// 회로 초기화 시 0 변수 생성
composer.zero_var = composer.add_witness_to_circuit_description(F::zero());
```

3.5.2 고정 변수 (Fixed Variables)

`add_witness_to_circuit_description` 메서드로 생성되는 변수들은 특정 값으로 고정됩니다:

```
pub fn add_witness_to_circuit_description(&mut self, value: F) -> Variable {
let var = self.add_input(value);
self.constrain_to_constant(var, value, None);
var
}
```

3.6 변수 생명주기

1. **생성**: `add_input()` 또는 `add_witness_to_circuit_description()` 을 통해 생성
2. **사용**: 다양한 게이트에서 와이어로 사용
3. **추적**: 순열 시스템에서 모든 사용 위치를 추적
4. **제약**: 게이트를 통해 다른 변수들과의 관계 정의
5. **검증**: 증명 생성 시 모든 제약 조건 확인

4. 게이트 구현 방식

StandardComposer는 다양한 게이트를 구현하기 위해 선택자 벡터 시스템을 사용합니다. 각 게이트는 특정 선택자 벡터의 값을 설정하여 구현되며, 모든 게이트는 통합된 제약 방정식을 따릅니다.

4.1 통합 제약 방정식

PLONK의 모든 게이트는 다음과 같은 통합 제약 방정식을 따릅니다:

```
q_arith * [
(a * b) * q_m + a * q_l + b * q_r + q_c + PI + q_o * c + q_4 * d +
q_hl * a^5 + q_hr * b^5 + q_h4 * d^5
] +
q_range * [range_constraint] +
q_logic * [logic_constraint] +
q_fixed_group_add * [fixed_ecc_constraint] +
q_variable_group_add * [variable_ecc_constraint] +
q_lookup * [lookup_constraint] = 0
```

여기서:

- `a`, `b`, `c`, `d` 는 4개의 와이어 값
- `q_*` 는 각각의 선택자 값
- `PI` 는 공개 입력 값

4.2 산술 게이트 (Arithmetic Gate)

산술 게이트는 `arithmetic_gate` 메서드를 통해 구현됩니다. 이 메서드는 `ArithmeticGate` 구조체를 사용하여 게이트의 속성을 설정합니다:

```
#[derive(Debug, Clone, Copy)]
```

```
pub struct ArithmeticGate<F>
where
F: PrimeField,
{
pub(crate) witness: Option<(Variable, Variable, Option<Variable>)>,
pub(crate) fan_in_3: Option<(F, Variable)>,
pub(crate) mul_selector: F,
pub(crate) add_selectors: (F, F),
pub(crate) out_selector: F,
pub(crate) const_selector: F,
pub(crate) pi: Option<F>,
}
```

산술 게이트는 빌더 패턴을 사용하여 구성됩니다:

```
// 덧셈 게이트: a + b = c
let c = composer.arithmetic_gate(|gate| {
gate.witness(a, b, None)
.add(F::one(), F::one())
});

// 곱셈 게이트: a * b = c
let c = composer.arithmetic_gate(|gate| {
```

```
gate.witness(a, b, None)

.mul(F::one())

});



// 복합 게이트: a * b + c * d + constant = output

let output = composer.arithmetic_gate(|gate| {

gate.witness(a, b, None)

.mul(F::from(2u64))

.add(F::one(), F::one())

.fan_in_3(F::from(3u64), d)

.constant(F::from(5u64))

});
```

4.3 다항식 게이트 (Polynomial Gate)

다항식 게이트는 가장 일반적인 형태의 게이트로, 사용자가 모든 선택자 값을 직접 지정할 수 있습니다:

```
pub fn poly_gate(

&mut self,

a: Variable,

b: Variable,

c: Variable,

q_m: F, // 곱셈 선택자

q_l: F, // 왼쪽 와이어 선택자
```

```
q_r: F, // 오른쪽 와이어 선택자

q_o: F, // 출력 와이어 선택자

q_c: F, // 상수 선택자

pi: Option<F>, // 공개 입력

) -> (Variable, Variable, Variable)
```

4.4 특수 게이트들

4.4.1 범위 게이트 (Range Gate)

변수가 특정 비트 수 범위 내에 있는지 확인합니다:

```
// a가 6비트 범위 내에 있는지 확인 (0 <= a < 2^6)

let  decomposition = composer.range_gate(a, 6);
```

4.4.2 불리언 게이트 (Boolean Gate)

변수가 0 또는 1인지 확인합니다:

```
// a가 0 또는 1인지 확인

let  boolean_var = composer.boolean_gate(a);
```

4.4.3 논리 게이트 (Logic Gate)

비트 단위 논리 연산을 수행합니다:

```
// a AND b

let  and_result = composer.logic_gate(a, b, LogicOp::And);
```

```
// a XOR b
let  xor_result = composer.logic_gate(a, b, LogicOp::Xor);
```

4.5 게이트 추가 과정

게이트를 추가하는 과정은 다음과 같은 단계를 따릅니다:

4.5.1 와이어 설정

```
// 4개의 와이어에 변수 할당
self.w_l.push(a); // 왼쪽 와이어
self.w_r.push(b); // 오른쪽 와이어
self.w_o.push(c); // 출력 와이어
self.w_4.push(d); // 네 번째 와이어 (또는 zero_var)
```

4.5.2 선택자 설정

```
// 산술 게이트 선택자 설정
self.q_m.push(q_m); // 곱셈 선택자
self.q_l.push(q_l); // 왼쪽 선택자
self.q_r.push(q_r); // 오른쪽 선택자
self.q_o.push(q_o); // 출력 선택자
self.q_c.push(q_c); // 상수 선택자
self.q_4.push(q_4); // 네 번째 와이어 선택자
```

```
// 게이트 타입 선택자 설정
self.q_arith.push(F::one()); // 산술 게이트 활성화
self.q_range.push(F::zero()); // 다른 게이트 타입 비활성화
self.q_logic.push(F::zero());
// ... 기타 선택자들
```

4.5.3 공개 입력 처리

```
if  let Some(pi) = pi {
self.add_pi(self.n, &pi)?;
}
```

4.5.4 순열 맵 업데이트

```
// 변수들을 현재 게이트의 와이어에 연결
self.perm.add_variables_to_map(a, b, c, d, self.n);
```

4.5.5 게이트 카운터 증가

```
self.n += 1;
```

4.6 게이트 최적화

StandardComposer는 다음과 같은 최적화 기법을 사용합니다:

1. **선택자 재사용**: 동일한 타입의 게이트들은 같은 선택자 패턴을 공유합니다.
2. **와이어 재사용**: 변수들은 여러 게이트에서 재사용될 수 있습니다.
3. **메모리 사전 할당**: `with_expected_size()` 를 통해 벡터 재할당을 최소화합니다.
4. **블라인딩 팩터**: 영지식 속성을 위한 랜덤 게이트들을 자동으로 추가합니다.

5. 실제 사용 예제

다음은 StandardComposer를 사용한 실제 회로 구성 예제입니다:

```
// 회로 초기화
let mut composer = StandardComposer::<BlsScalar, JubJubParameters>::new();

// 변수 추가
let a = composer.add_input(BlsScalar::from(20u64));
let b = composer.add_input(BlsScalar::from(5u64));
let zero = composer.zero_var();

// 제약 조건 1: a + b = c (공개 입력으로)
composer.arithmetic_gate(|gate| {
gate.witness(a, b, Some(zero))
.add(F::one(), F::one())
.pi(-BlsScalar::from(25u64)) // c = 25
});
```

```
// 제약 조건 2: a와 b의 범위 확인

composer.range_gate(a, 6); // a < 2^6

composer.range_gate(b, 4); // b < 2^4



// 제약 조건 3: a * b = d (공개 입력으로)

composer.arithmetic_gate(|gate| {

gate.witness(a, b, Some(zero))

.mul(F::one())

.pi(-BlsScalar::from(100u64)) // d = 100

});



// 타원 곡선 연산 예제

let  e = composer.add_input(from_embedded_curve_scalar(JubJubScalar::from(2u64))

let  generator = JubJubParameters::AFFINE_GENERATOR;

let  scalar_mul_result = composer.fixed_base_scalar_mul(e, generator);



// 결과를 공개 입력과 비교

composer.assert_equal_public_point(scalar_mul_result, expected_point);
```

6. 성능 고려사항

6.1 메모리 최적화

- `with_expected_size()` 를 사용하여 벡터 재할당을 최소화합니다.

- 변수 맵은 HashMap을 사용하여 O(1) 접근 시간을 제공합니다.
- 순열 시스템은 효율적인 변수 추적을 위해 최적화되어 있습니다.

6.2 회로 크기 최적화

- 게이트 수를 최소화하기 위해 복합 게이트를 사용합니다.
- 룩업 테이블을 활용하여 복잡한 연산을 효율화합니다.
- 불필요한 제약 조건을 피하고 게이트를 재사용합니다.

6.3 증명 생성 최적화

- 블라인딩 팩터가 자동으로 추가되어 영지식 속성을 보장합니다.
- 회로 크기는 2의 거듭제곱으로 패딩되어 FFT 효율성을 높입니다.

7. 결론

StandardComposer는 PLONK 회로를 구성하기 위한 강력하고 유연한 도구입니다. 주요 특징은 다음과 같습니다:

7.1 핵심 강점

1. **통합된 게이트 시스템**: 모든 게이트 타입이 통일된 제약 방정식을 따릅니다.
2. **효율적인 변수 관리**: 순열 시스템을 통한 체계적인 변수 추적
3. **유연한 회로 구성**: 다양한 게이트 타입과 제약 조건 지원
4. **성능 최적화**: 메모리 사전 할당과 효율적인 데이터 구조 사용
5. **사용자 친화적 API**: 빌더 패턴과 직관적인 메서드 제공

7.2 사용 시나리오

- **산술 연산**: 기본적인 덧셈, 곱셈, 상수 제약

- **범위 증명**: 변수가 특정 범위 내에 있음을 증명
- **논리 연산**: 비트 단위 AND, XOR 등의 연산
- **조건부 로직**: 조건에 따른 값 선택과 분기
- **타원 곡선 연산**: 스칼라 곱셈과 점 연산
- **해시 함수**: Poseidon 등의 암호학적 해시 함수

StandardComposer를 통해 개발자는 복잡한 영지식 증명 회로를 효율적으로 구성할 수 있으며, PLONK 증명 시스템의 모든 기능을 활용할 수 있습니다.